

Task 05 - Part 01: Question Answers

C# Fundamentals - Exceptions, Nullable Types, Arrays & Method Parameters

Q1. What is the purpose of the finally block?

The **finally** block contains code that is **guaranteed to execute** after a try block finishes, regardless of whether an exception was thrown or caught, and regardless of whether the try block completed normally, returned early, or the exception went unhandled. It's typically used for cleanup logic that must always run - such as closing a file, releasing a database connection, or (as in this case) printing a completion message - so that important cleanup steps are never skipped just because something went wrong.

Q2. How does `int.TryParse()` improve program robustness compared to `int.Parse()`?

`int.Parse()` throws an exception (`FormatException`, `ArgumentNullException`, or `OverflowException`) whenever the input isn't a valid, in-range integer, which means the calling code must wrap it in a try-catch or risk crashing the program. `int.TryParse()` instead attempts the conversion and returns a **bool** indicating whether it succeeded, placing the converted value (or 0 on failure) into an out parameter. This lets a program validate user input with a simple if-check instead of exception handling, avoiding the performance cost of thrown exceptions and making input-validation code cleaner and less error-prone - which is exactly why it's the preferred approach for defensive coding against untrusted input.

Q3. What exception occurs when trying to access `Value` on a null `Nullable<T>`?

Accessing the **Value** property on a `Nullable<T>` (e.g., `int?`) that currently holds no value (is null) throws an **InvalidOperationException**, typically with a message like "Nullable object must have a value." This is why it's best practice to check **HasValue** (or compare against null) before accessing `Value`, or to use the null-coalescing operator (`??`) to safely provide a fallback default instead of risking this exception.

Q4. Why is it necessary to check array bounds before accessing elements?

Arrays in C# have a fixed size, and accessing an index outside the valid range (less than 0 or greater than or equal to the array's `Length`) throws an **IndexOutOfRangeException** at runtime, which will crash the program if left unhandled. Checking bounds before accessing an element - for example, verifying that an index is within 0 to `array.Length - 1` - prevents this exception, allows the program to handle edge cases gracefully (such as user-provided indices), and avoids relying on try-catch purely as a substitute for basic validation logic.

Q5. How is the `GetLength(dimension)` method used in multi-dimensional arrays?

GetLength(dimension) returns the number of elements along a specific dimension of a multidimensional array, where the dimension is specified as a zero-based index - GetLength(0) returns the number of rows, GetLength(1) returns the number of columns (for a 2D array), and so on for higher dimensions. It's commonly used to control the bounds of nested loops when iterating over the array, since it lets each loop iterate exactly as many times as that dimension actually contains, regardless of the array's overall total element count.

Q6. How does the memory allocation differ between jagged arrays and rectangular arrays?

A **rectangular array** (e.g., int[,]) is allocated as a **single, contiguous block of memory** that stores all elements together, with every row guaranteed to have the same number of columns. A **jagged array** (e.g., int[][]) is actually an **array of arrays** - the outer array stores references (pointers) to separate, independently allocated inner arrays, and each inner array ("row") can have a completely different length. This makes jagged arrays more memory-flexible when row sizes vary significantly, since no space is wasted padding shorter rows to match the longest one, but it adds a small overhead from the extra array-of-references indirection that rectangular arrays don't have.

Q7. What is the purpose of nullable reference types in C#?

Nullable reference types (introduced in C# 8) let the compiler help catch potential **NullReferenceException** bugs at **compile time** rather than at runtime. When enabled, reference types (like string) are treated as non-nullable by default, and a type must be explicitly marked as nullable (string?) to indicate that null is a valid, expected value. The compiler then warns the developer whenever a possibly-null value is used without a null check, or when a non-nullable reference is assigned a value that might be null - making the intent around nullability explicit in the code and reducing an entire class of common runtime bugs.

Q8. What is the performance impact of boxing and unboxing in C#?

Boxing (converting a value type to an object/reference type) allocates a new object on the managed heap and copies the value type's data into it, while **unboxing** (converting back to the original value type) copies the data back out and also performs a runtime type check. Both operations add measurable overhead compared to working with the value type directly: extra heap allocations increase garbage-collector pressure, and the copying plus type-checking adds CPU cost. In performance-sensitive code - especially inside loops handling large numbers of values - frequent boxing and unboxing can noticeably degrade performance, which is why generic collections (like List<int> instead of ArrayList) are preferred, since generics avoid boxing value types entirely.

Q9. Why must out parameters be initialized inside the method?

An out parameter is a contract that guarantees the calling code will receive a definitely-assigned value once the method returns, since the caller isn't required to initialize the variable before passing it in. Because of this contract, the C# compiler requires that **every out parameter be assigned a value before the method returns** on every possible code path - if any path (like a branch or early return)

fails to assign it, the code will not compile. This enforced initialization prevents the caller from ever ending up with an unpredictable, garbage value in the out variable.

Q10. Why must optional parameters always appear at the end of a method's parameter list?

Optional parameters must come after all required parameters because C# resolves arguments to parameters **positionally** by default - the compiler matches each argument in a method call to a parameter based on its position, working left to right. If an optional parameter were placed before a required one, the compiler would have no unambiguous way to determine which arguments correspond to which parameters when some are omitted. Placing optional parameters at the end keeps the required parameters' positions fixed and unambiguous, letting the caller safely omit only the trailing optional ones (or use named parameters to skip around them).

Q11. How does the null propagation operator prevent NullReferenceException?

The null-conditional (null propagation) operator `?.` checks whether the object on its left side is null before attempting to access a member or call a method on it. If the object is null, the entire expression **short-circuits and evaluates to null** immediately, instead of throwing a `NullReferenceException`. This lets a chain of member accesses (e.g., `array?.Length` or `person?.Address?.City`) be written safely and concisely, without needing a separate explicit null check before every single step in the chain.

Q12. When is a switch expression preferred over a traditional if statement?

A switch expression is generally preferred when a single value needs to be mapped to one of several possible results based on discrete patterns or values - it's more concise than an equivalent chain of if-else statements, reads more like a direct mapping/lookup table, and (being an expression rather than a statement) can be used directly as a value, e.g., assigned straight into a variable or returned from a method. It becomes especially useful with pattern matching for filtering complex conditions. A traditional if statement remains more appropriate when the logic involves complex, multi-variable boolean conditions that don't reduce cleanly to matching a single value against a set of cases.

Q13. What are the limitations of the params keyword in method definitions?

A method can only have **one params parameter**, and it **must be the last parameter** in the parameter list, since the compiler needs to know exactly where the variable-length arguments begin. The params parameter must also be a **single-dimensional array type** (jagged arrays are technically allowed as the element type, but multidimensional arrays like `int[,]` cannot be used directly as params). Additionally, calling a params method with individually-listed arguments causes the compiler to allocate a new array behind the scenes on every call, which adds a small performance overhead compared to calling an overload that accepts a fixed number of parameters directly - something to be mindful of in performance-critical, frequently-called code.